BASTIDORES TÉCNICOS · OPENCLAW

A Evolução da Memória no OpenClaw

Como a IA aprendeu a lembrar — do zero até o sistema atual

Bruno Okamoto · Fundador, Comunidade Micro-SaaS PRO

Fevereiro 2026 · @microsaas

Quando você começa a usar um assistente de IA, uma das primeiras frustrações é: **ela esquece tudo quando a conversa termina.**

Mas o que acontece quando você resolve esse problema de forma sistemática? Você transforma uma IA genérica em uma assistente que evolui junto com você.

Esse documento conta a jornada de como eu construí e refinei o sistema de memória da Amora — minha assistente baseada em OpenClaw — desde o início até a versão atual.

CAPÍTULO 1

O Problema do Peixinho Dourado

A maioria das IAs tem o que chamo de "problema do peixinho dourado": cada conversa começa do zero.

Você explica seu negócio → ela entende → conversa termina → próxima conversa: explica tudo de novo.

Isso não é só inconveniente. É um bloqueador real para usar IA como parte do seu workflow de trabalho.

> ***O desafio central:*** *Como fazer a IA lembrar do que importa, sem ficar lenta e cara por carregar contexto infinito?*

CAPÍTULO 2

A Evolução em 4 Fases

FASE 1

Sem memória (Padrão)

"Cada conversa começa do zero"

O estado padrão de qualquer IA. Sem persistência. Sem histórico. Cada sessão é uma lousa em branco.

Problema: Você passa mais tempo explicando contexto do que resolvendo problemas.

FASE 2

Arquivos de contexto manuais

"Descrevi meu negócio em um arquivo"

A primeira solução: criar arquivos markdown que descrevem quem você é, seu negócio, seu tom de voz. O OpenClaw carrega esses arquivos no início de cada sessão.

Arquivos criados: `SOUL.md` (personalidade), `USER.md` (perfil), `AGENTS.md` (regras), `TOOLS.md` (integrações).

Evolução: A IA sabe quem você é. Mas ainda não lembra de conversas passadas ou decisões tomadas.

FASE 3

Topic Files + Memory estruturada

"Organizei a memória em arquivos temáticos"

A segunda evolução: criar um sistema de memória organizado por domínio.

```
memory/
├── decisions.md     ← Decisões tomadas (com contexto e data)
├── projects.md      ← Projetos ativos
├── people.md        ← Equipe e contatos
├── pending.md       ← Aguardando input
├── lessons.md       ← Aprendizados
└── YYYY-MM-DD.md    ← Notas diárias (descartadas após 14 dias)
```

O que mudou: A Amora consulta esses arquivos antes de responder. Pergunta "qual foi a decisão sobre X?" → ela busca em `decisions.md`.

Problema que surgiu: A busca era só semântica — perdia termos exatos. E a memória sumia cedo demais em conversas longas.

FASE 4 · ATUAL ✨

Sistema atual — Memória Inteligente

"Configuração atual — Fevereiro 2026"

A versão atual resolve os problemas das fases anteriores com **5 melhorias simultâneas**. Detalhadas no próximo capítulo.

CAPÍTULO 3

As 5 Melhorias da Fase 4

Configuração atual — Fevereiro 2026

1 Memory Flush: 5k → 40k tokens

❌ ANTES

"Esquecia" partes da conversa ao atingir 5.000 tokens — equivale a ~15-20 minutos de papo.

✅ DEPOIS

Threshold de 40.000 tokens. Mantém muito mais contexto ativo antes de compactar.

Na prática: Reuniões longas, debugging, planejamento extenso — tudo fica na memória ativa por muito mais tempo.

2 Flush Prompt Personalizado

❌ ANTES

Prompt genérico — salvava "o que parecia importante" sem critérios claros.

✅ DEPOIS

5 categorias: Decisões · Mudanças · Lições · Bloqueios · Fatos-chave.

Na prática: Nada importante se perde. Antes de compactar, a Amora salva tudo nos arquivos corretos.

3 Busca Híbrida: BM25 + Vector (50/50)

A mudança mais técnica, mas com maior impacto na qualidade das respostas.

BM25 (NOVO)

Busca por palavras-chave exatas.
Como o Google clássico.
Ótimo para nomes, datas, termos específicos.

VECTOR (JÁ EXISTIA)

Busca por similaridade semântica.
Encontra coisas com sentido parecido.
Ótimo para conceitos e contexto.

Exemplo: "Qual foi a decisão sobre o Metricaas em janeiro?" — BM25 pega "Metricaas" e "janeiro" exatamente. Vector pega o contexto de decisão. Juntos: resposta muito mais precisa.

4 Fontes de busca: memory + sessions

❌ ANTES

Buscava só nos arquivos de memória (`memory/*.md`).

✅ DEPOIS

Busca também no histórico de sessões — conversas antigas ficam indexadas.

Na prática: Coisas mencionadas em conversas passadas (mesmo sem virar arquivo) agora podem ser recuperadas.

5 Context Pruning TTL: 30 min → 6 horas

❌ ANTES

Contexto "podado" após 30 min de inatividade. Voltava e o fio estava perdido.

✅ DEPOIS

TTL de 6h + `keepLastAssistants: 3` (mantém as 3 últimas respostas sempre ativas).

Na prática: Você pode voltar à conversa horas depois e a Amora ainda lembra do que estava sendo discutido.

CAPÍTULO 4

Por que isso importa para você

Se você usa IA como ferramenta de trabalho, a memória não é um detalhe técnico — é o que separa uma IA "assistente genérica" de uma IA que **conhece seu negócio**.

SITUAÇÃO	SEM MEMÓRIA	COM MEMÓRIA (ATUAL)
Perguntar sobre um projeto	Precisa explicar tudo de novo	Amora já sabe o contexto
Decisão tomada há 3 semanas	Esquecida	Registrada com data e contexto
Conversa longa (2h+)	Começa a "esquecer" partes	Mantém contexto (40k tokens)
Volta à conversa 4h depois	Contexto perdido	TTL de 6h mantém tudo
Buscar "a decisão sobre X"	Encontra coisas parecidas	Encontra exatamente "X" (BM25)

CAPÍTULO 5

Como implementar: configuração atual

Se você usa OpenClaw e quer replicar essa configuração, aqui está o bloco exato do `openclaw.json`:

```
{
  "memorySearch": {
    "sources": ["memory", "sessions"],
    "query": {
      "hybrid": {
        "enabled": true,
        "vectorWeight": 0.5,
        "textWeight": 0.5
      }
    }
  },
  "contextPruning": {
    "mode": "cache-ttl",
    "ttl": "6h",
    "keepLastAssistants": 3
  },
```

```
  "compaction": {
    "mode": "default",
    "reserveTokensFloor": 30000,
    "memoryFlush": {
      "enabled": true,
      "softThresholdTokens": 40000,
      "prompt": "Before this context is compacted, extract and save all
important information to memory files. Focus specifically on:
1. DECISIONS made (what was decided and why)
2. STATE CHANGES (what changed, what was configured, what was installed)
3. LESSONS LEARNED (what worked, what didn't, insights)
4. BLOCKERS or pending items waiting for input
5. Key facts about projects, people, or systems discussed

Write to the appropriate memory files in /workspace/memory/ and
/shared/ as needed. Do not skip — missing context is permanently
lost after compaction."
    }
  }
}
```

Resumo: evolução completa

1. **Fase 1 — Sem memória**
   Estado padrão. Cada conversa começa do zero.
2. **Fase 2 — Arquivos de contexto estáticos**
   SOUL.md, USER.md, AGENTS.md. A IA sabe quem você é.
3. **Fase 3 — Topic files + memória estruturada**
   memory/*.md por domínio. Decisões, projetos, pessoas, lições.
4. **Fase 4 — Sistema atual (Fev 2026) ✨**
   Busca híbrida BM25+vector · TTL 6h · Flush 40k tokens · Prompt customizado · Histórico de sessões.

Conclusão

Memória não é feature — é infraestrutura.

A diferença entre uma IA que você usa uma vez e abandona vs. uma que vira parte do seu workflow diário é exatamente isso: **ela lembra de você**.

Cada melhoria nesse sistema foi uma resposta a uma frustração real. Se você está construindo seu próprio setup de IA, espero que essa jornada te poupe pelo menos algumas das pedradas que eu levei.

Bruno Okamoto · Fundador, Comunidade Micro-SaaS PRO

@microsaas · microsaas.com.br

Quer mais bastidores reais? Canal do YouTube @microsaas e Comunidade Micro-SaaS PRO.